

SOPORTE DEL JUEGO BUSCAMINAS- PHYTON

VIERHYS SANDRITH ABELLO FLOREZ

OBJETIVOS:

El objetivo principal del proyecto es recrear el clásico juego “Buscaminas” en Python con los conceptos previos vistos en clases, permitiendo que el jugador pueda descubrir casillas seguras y evitar pisar minas, para esto el programa analizará la posición del jugador y revelará el número de minas cerca al jugador, pero este no sabrá específicamente donde estarán. El programa brindará la información del jugador como su nombre de usuario, su ID, el modo de nivel y el tiempo que tomó perder o ganar la partida y un ranking para los jugadores que escojan el modo “Expert”.

FUNCIONES PRINCIPALES:

def crear_usuario(): solicita los datos al usuario como su nombre, el nombre de usuario y genera un ID para el usuario.

def escoger_nivel(): brinda opciones para seleccionar (easy, medium, expert). Cada nivel tiene un tamaño de tablero y una cantidad de minas.

def inicio_tablero(filas, columnas): Crea un tablero (matriz) lleno de “-” el cual representará las casillas sin descubrir por el jugador.

def colocar_minas(tablero, minas): De manera aleatoria el programa pondrá el número de minas correspondiente al nivel seleccionado.

def contar_minas(tablero, fila, columna): Dependiendo de la posición del jugador esta funcion contará el número de minas al rededor.

def ver_tablero(tablero): Mostrará el estado actual del tablero del jugador

def inicio_juego(nombre, usuario_id, filas, columnas, minas, nivel): Muestra el flujo del juego, detecta las minas y se analiza si se finaliza o no el juego.

def ordenar_partidas(partidas): Ordena las partidas ganadas en modo “Expert” dependiendo del tiempo y se ordenará de menor a mayor.

def main(): Uso de modularidad, controla cómo va el funcionamiento del juego, registra los datos y recrea el ranking de los jugadores.

PROBLEMAS ENCONTRADOS:

1. No colocaba minas en el tablero verdadero por lo que el jugador ganaba al tener más espacios seguros

Antes de corregir:

```
def inicio_juego(nombre, correo, id, filas, columnas, minas):
    tablero_verdadero = inicio_tablero(filas, columnas) #mismo tablero para jugar
    tablero_jugador = inicio_tablero(filas, columnas)

36 def colocar_minas(tablero, minas):
37     t_filas = len(tablero)
38     t_columnas = len(tablero[0])
39     minas_colocadas = 0
40     while minas_colocadas < minas:
41         f = random.randint(0, t_filas - 1)
42         c = random.randint(0, t_columnas - 1)
43         if tablero[f][c] != "MINA": #si no hay mina se le pone una
44             tablero[f][c] = "MINA"
45             minas_colocadas += 1
46
```

Después de corregir:

```
def inicio_juego(nombre, usuario_id, filas, columnas, minas, nivel):
    """recopilacion de los datos y se mostrará el flujo del juego"""
    tablero_verdadero = inicio_tablero(filas, columnas)
    tablero_jugador = inicio_tablero(filas, columnas)
    colocar_minas(tablero_verdadero, minas)
```

```

43 def colocar_minas(tablero, minas):
44     """poner minas aleatorias en el tablero dependiendo del mood"""
45     t_fila = len(tablero)
46     t_columna = len(tablero[0])
47     minas_colocadas = 0
48     while minas_colocadas < minas:
49         f = random.randint(0, t_fila - 1)
50         c = random.randint(0, t_columna - 1)
51         if tablero[f][c] != "MINA":
52             tablero[f][c] = "MINA"
53             minas_colocadas += 1
54

```

2. Si el jugador ingresaba un movimiento que estaba fuera del tablero imprimiría un mensaje advirtiéndolo que el movimiento no es posible y cerraría el programa, pero no sucedía y se seguía ejecutando la jugada.
3. La función principal main() corre todo de forma lineal y no había posibilidad de jugar o almacenar información de los juegos anteriores.
4. La lógica en la estructura del juego estaba muy desordenada por lo que errores simples llenaban todo el programa de sub-errores que no debían estar.
5. El uso repetitivo de varias palabras como funciones confundía demasiado al programa al momento de ejecutar el código.
6. Usaba ver_fila de una manera muy innecesaria que solo alargaba el código y no ejecutaba correctamente.

Antes de corregir:

```

def ver_tablero(tablero):
    for fila in tablero:
        ver_fila= [] # Lista que se llenará con los valores de la fila
        for casilla in fila:
            ver_fila.append(casilla)
        for elemento in ver_fila:
            print(elemento, end=" ") # imprime la línea con espacio
        print() #vacío y pasa a la siguiente fila

```

Después de corregir:

```
def ver_tablero(tablero):  
    """Estado actual del tablero"""  
    for fila in tablero:  
        for elemento in fila:  
            print(elemento, end=" ")  
        print()
```

CONCLUSIONES:

La versión antigua presentaba errores como la ausencia de minas reales en el tablero, lo que afectaba la lógica del juego, además de una estructura repetitiva y la falta de control de errores. En resumen, a pesar de tener muchos desafíos a la hora de programar el juego este obtuvo avance sólido hacia un juego funcional usando estructuras fundamentales que se ha visto en clases y con un control de flujo y diseño enfocado en el usuario. Aunque se puedan realizar mejoras fuertes en el programa .